

Burp Suite Sequencer

The **Sequencer** is a tool inside Burp Suite used to test the **quality of randomness** in tokens or session IDs generated by a web application.

◆ **Why is Sequencer important?**

- Many applications generate **session tokens, CSRF tokens, or API keys** to identify users securely.
- If these tokens are **predictable** (weak randomness), attackers can **guess them** and hijack sessions.
- Sequencer helps a **SOC analyst / penetration tester** measure if the randomness is strong enough.

◆ **How Sequencer Works**

1. Capture the Token

- Use Burp Proxy or Repeater to capture a request that returns a token (e.g., session cookie, CSRF token).

2. Send to Sequencer

- Right-click → *Send to Sequencer*.

3. Start Capturing

- Sequencer will make repeated requests to the server and collect **many token samples**.

4. Statistical Analysis

- Sequencer performs **entropy tests** and **randomness analysis** on the collected tokens.
- It checks if the characters follow a predictable pattern.

◆ **Results in Sequencer**

- **Entropy** → Measures unpredictability (higher entropy = stronger token).
- **Bit Distribution** → Checks if bits are evenly distributed.
- **Character Distribution** → Ensures each character appears randomly.
- **Chi-Square Tests** → Mathematical test to see if sequence is truly random.
- **Overall Strength** → Burp gives a score like *“Poor”*, *“Acceptable”*, or *“Excellent”* randomness.

◆ **Example**

Let's say a web app generates session IDs like:

ABC1001

ABC1002

ABC1003

👉 Sequencer will easily detect this **pattern** → randomness is **poor** → vulnerable to session hijacking.

But if session IDs look like:

X7fj29LQa9mP0ZbY

U9ka3V7pQwL0XhMn

👉 Sequencer shows **high entropy** → good randomness → secure.

◆ **Where Sequencer is used in real-world?**

- Checking **Session IDs**.
- Checking **CSRF tokens**.
- Checking **Password reset tokens / API keys**.